

Blua — Sistema de Gerenciamento de Fila de Atendimento

Relatório Técnico · Sprint: Estruturas de Dados Aplicadas à Saúde Digital

Integrantes

Isabela Marques de Oliveira	567230
Isabelle Ramos De Filippis	566783
Joao Vitor Anunciacao Oliveira	567539
Paulo Ribeiro Marinho	567459
Samy Tamires de Sousa Cruz	566674

1. Introducao e Contexto do Problema

A Care Plus expandiu a plataforma Blua para oferecer cuidado remoto proativo. O desafio central e gerenciar, de forma eficiente e segura, o fluxo de atendimentos remotos: solicitacoes de teleconsulta chegam continuamente, medicos precisam atender pacientes na ordem correta e alertas criticos de saude devem ser priorizados imediatamente.

Para resolver esse problema foram escolhidas duas estruturas classicas: **Fila (FIFO)** e **Pilha (LIFO)**, implementadas do zero com listas encadeadas em C, sem uso de bibliotecas prontas. O arquivo principal e **main.c**, compilado com gcc (C99) e executavel tambem pelo OnlineGDB.

2. Estruturas de Dados Escolhidas

2.1 Fila FIFO - Pacientes Normais

A Fila (First In, First Out) garante que o primeiro paciente a solicitar uma teleconsulta seja o primeiro a ser atendido. Cada no da lista encadeada armazena os dados do paciente (struct Paciente) e um ponteiro **prox** para o proximo. A struct Fila mantem dois ponteiros: **inicio** (para dequeue) e **fim** (para enqueue), garantindo O(1) em ambas as operacoes.

Trecho do codigo — struct Fila:

```
typedef struct { NoPaciente *inicio; NoPaciente *fim; int tamanho; } Fila;
```

2.2 Fila de Prioridade - Emergencias

Uma segunda fila FIFO identica em estrutura foi criada exclusivamente para emergencias (prioridade = 1, definida via #define EMERGENCIA 1). A funcao **atender()** sempre verifica e esvazia a fila de emergencias antes de processar a fila normal, garantindo prioridade sem reordenacao — custo de enqueue continua O(1).

2.3 Pilha LIFO - Historico de Atendimentos

A Pilha (Last In, First Out) registra as ultimas MAX_HISTORICO (= 5) consultas finalizadas. O atendimento mais recente fica no topo, facilitando auditoria e a operacao de desfazer (pop). Quando a capacidade maxima e atingida, o push percorre a lista ate o fundo e descarta o registro mais antigo antes de inserir o novo no topo.

Trecho do código — struct Pilha:

```
typedef struct { NoPilha *topo; int tamanho; } Pilha;
```

3. Diagrama de Funcionamento

```
Paciente solicita teleconsulta
      |
      | Emergencia? (prioridade == 1)
      /      \
SIM           NAO
enqueue() enqueue()
fila_emerg fila_normal
      \      /
      |      |
      |      | atender() - fila_emerg primeiro
      |      |
      |      | dequeue() - remove paciente
      |      |
      |      | push() - historico (Pilha)
```

4. Analise de Complexidade

enqueue / push	O(1) — insere no fim	O(1)* — insere no topo
dequeue / pop	O(1) — remove do inicio	O(1) — remove do topo
peek	O(1) — acessa inicio	O(1) — acessa topo
fila_vazia / pilha_vazia	O(1)	O(1)
imprimir tudo	O(n)	O(n)

* Quando a pilha atinge MAX_HISTORICO, o push percorre a lista para remover o elemento mais antigo (O(n)). Com capacidade fixa e pequena (n=5), esse custo e negligenciavel na pratica.

5. Cenarios de Uso Demonstrados

Cenario 1 — Fila normal e atendimento sequencial

Enfileira 3 pacientes normais (Ana, Bruno, Carla) via enqueue() e processa 2 atendimentos com dequeue(). Demonstra que a ordem FIFO e respeitada e que os registros sao empilhados no historico via push().

Cenario 2 — Emergencia tem prioridade

Adiciona 2 pacientes normais (Diego, Eva) e 1 emergencia (Fabio — dor no peito). A funcao atender() checa a fila de emergencias primeiro: Fabio e atendido antes dos demais, independentemente da ordem de chegada.

Cenario 3 — Desfazer e capacidade da pilha

Executa pop() no historico simulando cancelamento ou reatribuicao do ultimo atendimento. Em seguida, insere 7 registros em uma pilha de capacidade 5, mostrando o descarte automatico dos mais antigos.

6. Justificativa das Escolhas

A **lista encadeada simples** foi preferida a arrays porque elimina a necessidade de realocacao de memoria e nao desperdiça espaco pre-alocado. Em ambiente hospitalar, onde o numero de pacientes simultaneos e imprevisivel, essa flexibilidade e importante.

A separacao em **duas filas** (normal e emergencia) mantem o custo de enqueue em $O(1)$ — uma unica fila ordenada por prioridade exigiria $O(n)$ para insercao. A funcao atender() resolve a prioridade no momento do despacho, sem custo adicional.

A **pilha com capacidade limitada** ($MAX_HISTORICO = 5$) evita crescimento ilimitado de memoria, descartando automaticamente os registros mais antigos quando cheia — comportamento adequado para auditoria de curto prazo e para a operacao de desfazer do ultimo atendimento.